

Kazakh-British Technical University

DATABASE

Laboratory Work №1

ER Diagram

Student Name: Saken Aibar

ID: 25B030240

09.09.2026

Almaty

1. System Description

The purpose of this database is to manage university students, courses, instructors, and course registrations.

Students can register for multiple courses, and each course can have multiple students. The `Registration` entity is used to represent this many-to-many relationship and stores additional information such as the registration date and the student's grade.

Each course is taught by exactly one instructor, while an instructor can teach multiple courses.

2. Entities and Attributes

Student

The `Student` entity stores information about university students.

<code>student_id</code>	INTEGER	PRIMARY KEY
<code>name</code>	VARCHAR(100)	NOT NULL
<code>email</code>	VARCHAR(255)	UNIQUE NOT NULL
<code>date_of_birth</code>	DATE	NOT NULL

`student_id` uniquely identifies each student. The student's name, email, and date of birth are required. The `email` attribute is unique to prevent multiple students from using the same email address.

Instructor

The `Instructor` entity stores information about university instructors.

<code>instructor_id</code>	INTEGER	PRIMARY KEY
<code>name</code>	VARCHAR(100)	NOT NULL
<code>email</code>	VARCHAR(255)	UNIQUE NOT NULL
<code>department</code>	VARCHAR(255)	NOT NULL

`instructor_id` uniquely identifies each instructor. The instructor's name, email, and department are required. The `email` attribute is unique.

Course

The `Course` entity stores information about university courses.

<code>course_id</code>	INTEGER	PRIMARY KEY
<code>course_name</code>	VARCHAR(100)	NOT NULL
<code>credits</code>	INTEGER	NOT NULL

`instructor_id` `INTEGER` `NOT NULL FOREIGN KEY`

`course_id` uniquely identifies each course. The course name and number of credits are required.

`instructor_id` is a foreign key that references `Instructor(instructor_id)`. It is `NOT NULL` because each course must be taught by exactly one instructor.

Registration

The `Registration` entity represents the registration of a student for a course.

<code>student_id</code>	<code>INTEGER</code>	<code>PRIMARY KEY, FOREIGN KEY, NOT NULL</code>
<code>course_id</code>	<code>INTEGER</code>	<code>PRIMARY KEY, FOREIGN KEY, NOT NULL</code>
<code>registration_date</code>	<code>DATE</code>	<code>NOT NULL</code>
<code>grade</code>	<code>INTEGER</code>	<code>NOT NULL</code>

`student_id` references `Student(student_id)`, while `course_id` references `Course(course_id)`.

The combination of `student_id` and `course_id` forms a **composite primary key**:

`PRIMARY KEY (student_id, course_id)`

This prevents the same student from being registered for the same course more than once.

3. Relationships

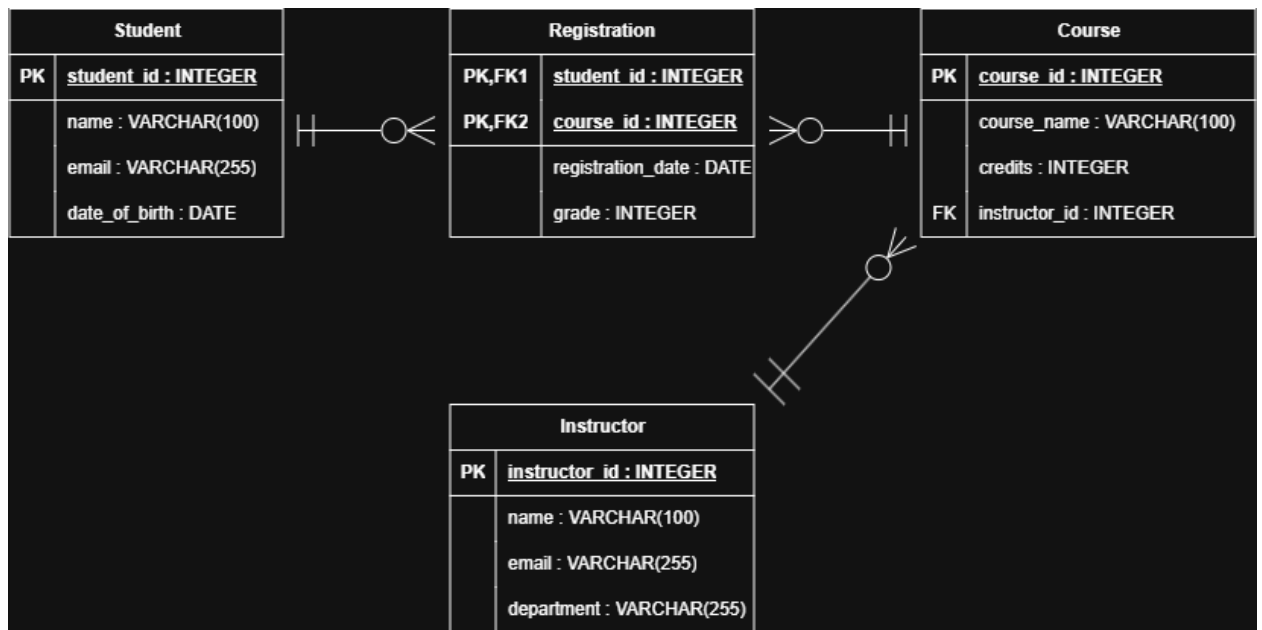
The relationship between `Student` and `Registration` is **one-to-many**. One student can have zero or many registrations, while each registration belongs to exactly one student.

The relationship between `Course` and `Registration` is **one-to-many**. One course can have zero or many registrations, while each registration belongs to exactly one course.

Therefore, the relationship between `Student` and `Course` is **many-to-many**, implemented through the `Registration` entity.

The relationship between `Instructor` and `Course` is **one-to-many**. One instructor can teach zero or many courses, while each course is taught by exactly one instructor.

4. ER Diagram



5. Legend

- PK — Primary Key
- FK — Foreign Key
- || — Exactly one
- O< — Zero or many
- **Crow's Foot symbol** — Represents the “many” side of a relationship